Speaking notes for the live demonstration

These notes go with `app/poker-solver.html`, the River Solver Table. The demonstration replaces the slides that used to show static strategy grids, and it takes about twelve minutes. A five minute version is at the end.

Every number quoted below is what the app shows for the talk example after the default 1 000 iterations of DCFR. The numbers on your screen can differ in the last digit, because the solve stops at a finite accuracy.

Before the talk

1. Open `app/poker-solver.html` in a browser. It is a single file and needs no network connection. The published copy on claude.ai is the same page.
2. Wait about two seconds. The page loads the river spot from the slides and solves it with DCFR on its own. The status line under the Solve button should read "DCFR after 1,000 iterations is within 0.0023% of the pot of an equilibrium".
3. Set the browser zoom so that the table, the grid and the top of the game tree are all visible. On a 1080p projector, 110% works well. Use the browser's full screen mode.
4. Open the **Demo guide** with the button at the top right. Each step in the guide has a **Show me** button that sets the page up for that step, so you never have to hunt for a control while speaking. The steps are in the same order as these notes. Close the guide when you want the whole screen.
5. If something goes wrong during the demonstration, press **Load the talk example** in the setup panel. It restores everything in two seconds.

What is on the screen

Say this once, before step 1, while pointing at each of the four panels.

- **The table** (top left) is the picture from the hold'em slide, drawn as a real table. The five board cards are in the middle, the big blind sits on the left and acts first, and the button sits on the right. The pot and the stacks update as the hand is played. The row of buttons under the table shows the actions that are allowed right now, with how often the solved strategy takes each one.
- **The range grid** (top right) is the 13 by 13 grid from the ranges slide. There is one tab for each player. For the player who is about to act, each square is coloured by action: green is check or call, blue is fold, and the reds are bets, darker for bigger bets. For the other player, the square shows how much of that hand is still in the range.
- **The game tree** (bottom left) is the public tree from the slides. Blue points are the big blind's decisions and orange points are the button's. The ring shows where the table is now, and thick branches are played often.
- **The solver** (bottom right) runs the algorithms and plots how far the current strategy is from an equilibrium.

Step 1. The spot from the slides (1 minute)

Do. Guide step 1, or just start from the freshly loaded page.

Say. "This is the river spot from the slides. The board is king, seven, two, ten, four. There are 100 chips in the pot and each player has 100 behind. The big blind arrives with 473 possible hands and the button with 328. Neither player knows the other's cards, so each player has to choose an action for every hand in the range at the same time. That whole table of choices is the unknown we are solving for."

Point at. The header line, then the two tabs of the grid. Click the Button tab and back, so the audience sees that the two ranges are different. The button has the strong hands in the top left corner, aces, kings and ace-king, and the big blind does not, because the big blind would have raised with them before the flop.

The math. A strategy is a vector with one probability for each hand and action at each decision point. This tree has 12 decision points and 21 endings, which gives about 17 600 unknowns in the sequence form.

Step 2. Race the four algorithms (2 minutes)

Do. Guide step 2, or press **Race all four**. The race takes about five seconds.

Say while it runs. "All four methods are the same loop. Each iteration walks the tree once for each player, computes a regret for every hand and action, and updates the strategy by regret matching. They differ only in how they weight old regrets and how they average the strategies. The vertical axis is the exploitability of the average strategy as a percentage of the pot, which is how much a perfect opponent could win against it. Both axes are logarithmic, so a straight line is a power law."

What appears. After 1 000 iterations the table under the chart reads approximately:

algorithm	exploitability, percent of pot
CFR	0.11
PCFR+	0.009
CFR+	0.007
DCFR	0.002

Say after it finishes. "Plain CFR is clearly the slowest. The theorem guarantees a rate of one over the square root of T for all four methods, and the three modern variants do much better than that bound in practice. DCFR is about fifty times more accurate than CFR after the same work. This is why every practical solver uses one of these variants. Notice also that even the best curve is still a straight line on this plot. That sublinear tail is the reason for the second half of the talk."

Extra, if there is time. Hover over the chart to read values at a given iteration. Set the iterations box to 3000 and press Race all four again to extend the curves. While a solve runs, the grid changes live, so the audience can watch the strategy form.

If asked why PCFR+ is not the best here. Farina, Kroer and Sandholm report the same pattern in the paper that introduced it: it is the strongest method on many non-poker games, and DCFR stays ahead on poker endgames like this one.

Step 3. The big blind's opening strategy (1.5 minutes)

Do. Guide step 3. The table is at the start of the river and the Big blind tab is selected.

Say. "Here is the solution for the first decision. Over the whole range, the big blind checks about 82% of the time, bets a third of the pot about 16%, and bets three quarters of the pot about 2%. The all-in is never used. Now look at which hands do what. The medium strength hands are solid green, they always check. The betting hands are of two kinds. The first kind is hands that expect to be ahead when called: a pair of kings with a good second card, such as king-queen and king-jack, and the pairs of tens. The second kind is hands with no pair at all, such as queen-jack and jack-nine, which bet part of the

time as bluffs. Nobody told the solver about bluffing. It appears because without the bluffs the button could simply fold every time the big blind bets."

Point at. The coloured bar under the grid, which is the whole range split by action, and the list on the right with the value of each action. Hover over the KQs square: the panel on the right lists each individual king-queen hand, with its strategy and its value in chips.

The math. Many squares show two colours. A hand that mixes must have the same value for both actions. That is complementary slackness for the linear program, and step 6 shows it with numbers.

Step 4. One hand at a time (1 minute)

Do. Guide step 4. The big blind is given a pair of sevens, which makes three of a kind with the seven on the board, and the button is given king-queen of hearts. The cards appear face up on the table.

Say. "The grid shows the strategy for every hand. The table shows one deal. With three sevens, the big blind checks every time, even though this is nearly the best possible hand. The action buttons show why: checking is worth about 145 chips and betting is worth about 142. The solver is setting a trap."

Point at. The lines "this hand 100.0%" and the values on the action buttons.

Extra. Click the hidden cards of either player to choose any two cards, or press **Deal hands from the ranges** for a random deal.

Step 5. Follow a line through the tree (1.5 minutes)

Do. Guide step 5. The big blind checks and the button bets 75. Alternatively click Check and then Bet 75 yourself, which shows the audience that the page is a real game.

Say. "The big blind has checked and the button has bet 75. Three things changed on the screen. On the table, the pot is now 175 and the big blind has to put in 75 to call. In the tree, the ring moved two steps along the highlighted path. And the grid now shows the big blind's answer to the bet: fold 63% of the time, call 20%, and raise all-in 17%. The three sevens that checked a moment ago are now raising all-in every time, which is the trap closing."

Point at. The Button tab now says about 20 hands. Click it. "The button only makes this bet with 20 of its 328 hands, and the grid shows which ones. The big blind's answer is computed against exactly this narrow range."

The math. The numbers on each branch of the tree are conditional probabilities, and the product along a path is the reach probability that weights the regrets in CFR. Press **Current line** above the tree to fold away the branches that were not taken.

Step 6. A hand that is indifferent (1.5 minutes)

Do. Guide step 6. The page selects ace-nine offsuit, a hand that mixes between folding and calling.

Say. "Ace-nine folds 63% of the time and calls 37%. Look at the values in the small table on the right. Folding is worth 0.0 chips and calling is worth 0.0 chips, or 0.1 after only 1 000 iterations, which is the accuracy of the solve. Raising would lose 2 chips, so it is never used. This is the equilibrium condition from the linear programming slide, seen on one hand. An action can only be used with positive probability if its value equals the best value. Every mixed square on this grid is an equation of that kind, and the solver has found the frequencies that make all of them hold at once."

Extra. Click any other two-coloured square to see the same equality. Squares with one colour have a strict inequality instead.

Step 7. The pot odds check (1.5 minutes)

Do. Guide step 7. The big blind bets 33 and the Button tab is selected.

Say. "Earlier we derived a formula on the toy game. Against a bet of size s times the pot, the defender has to continue with probability one over one plus s, otherwise bluffing with any two cards is profitable. For a bet of one third of the pot the formula says: fold at most 24.8%. The solved strategy for the button folds 26%. The solver was never given this formula. It comes out of the indifference of the big blind's bluffing hands."

Then. Click **Back to the start**, then Check, then All-in 100. "Now the roles are reversed and the button has bet the size of the pot. The formula says the big blind should fold half the time. The solver folds 68%. The formula assumed that a bluffing hand wins nothing when it checks. Here the button's bluffs do win sometimes when they check, so they need more folds to be worth betting. The formula is a special case, and the solver tells us how far off it is."

Step 8. The button's side (1 minute)

Do. Guide step 8. The big blind has checked and the Button tab is selected.

Say. "After a check the button checks back about half its range and moves all-in with about 45%. The all-in hands are of two kinds. The top left corner is strong hands, kings and better, betting for value. The scattered dark squares lower down, such as queen-jack, jack-nine and six-five suited, are hands with no pair, and they are bluffs. The green squares, the ace-high hands and the middle pairs, are good enough to win some showdowns but not good enough to bet, so they check."

The math. This is a polarised range, strong hands and bluffs with nothing in between, and it is the same structure as the toy game solution.

Step 9. Change the river card (1 minute)

Do. Guide step 9. The last board card becomes the nine of hearts and the spot is solved again in about two seconds. You can also click any board card and choose a replacement.

Say. "A solver is a function from the public situation to a strategy. I change one card and solve again. With a nine on the river, queen-jack is now a straight. The big blind's share of the pot goes from 33.8 to 35.6 chips, the three quarter pot bet disappears, and the betting squares move to different hands."

Step 10. Change a range (1 minute)

Do. Guide step 10. The range editor opens. Choose a brush weight, then click or drag over squares. Add ace-ace and king-king to the big blind, for example. The solve restarts when you release the mouse.

Say. "The ranges are inputs. In practice they come from solving the earlier streets, and a small change in them changes the answer. Here I give the big blind the strong pairs it did not have. Click Check and look at the Button tab to see how the button's answer to a check has changed."

Extra. The text box under the brushes takes standard range notation, for example `TT+, AQs+, 76s:0.5`.

Step 11. Change the bet sizes (1 minute)

Do. Guide step 11. In the setup panel, change the first player's bet sizes to `50, 125`, or raise the maximum number of bets and raises to 3.

Say. "The bet sizes are a modelling choice. The real game allows any size, and the solver only sees the sizes we put in the tree. More sizes give a bigger tree, more unknowns and more work for each iteration. Choosing a small tree that loses little value is its own research topic."

Close the demonstration. "Everything on this page runs in the browser in a few hundred lines of JavaScript, and one iteration takes about a millisecond. The first-order methods get to a tenth of a percent of the pot very quickly. The rest of the talk is about what to do when we want twelve digits."

Five minute version

1. Step 1 in thirty seconds: the table, the two ranges, the tree.
2. Step 2: race the four algorithms and read the final table.
3. Step 3: the opening strategy, value bets and bluffs.
4. Step 6: the indifferent hand, with the two equal values.
5. Step 7, first half only: the pot odds formula against the solved fold frequency.

Questions that the demonstration tends to raise

- **Is the strategy on the screen exact?** No. It is the average strategy after a fixed number of iterations, and the status line states its exploitability. At 1 000 iterations of DCFR it is within 0.002% of the pot.
- **Is the equilibrium unique?** The value of the game is unique, 33.8 chips for the big blind here. The strategies are not. Hands that are never reached, and exact ties between hands, leave freedom. This is the flatness that makes the second-order method need its restriction step.
- **Why do the numbers change slightly between runs of different algorithms?** Each algorithm stops at a different approximate equilibrium, and the equilibrium set is not a single point.
- **Could this run for the whole game instead of one river?** The same loop runs on earlier streets, with chance nodes for the cards still to come. The tree is then millions of times larger, which is why production solvers use abstraction and a lot of memory.
- **What does "value" mean in the panels?** It is the expected number of chips the hand wins by the end, counted from the start of the river, when both players follow the strategy shown. Folding immediately is worth 0 and winning the starting pot uncontested is worth 100.
- **How does the page compute the best response for the exploitability?** One tree walk for each player. At the showdowns it sorts the hands by strength once and uses running sums, with a correction for hands that share a card, so each ending costs time proportional to the number of hands.

Controls, for reference

control	what it does
Action buttons under the table	play that action and move down the tree
Back one action, Back to the start	move up the tree
Let the solver act	sample an action from the solved strategy, using the dealt hand if there is one

control	what it does
Deal hands from the ranges	deal both players a random hand and return to the start
Click a board card	replace it, then solve again
Click a player's cards	choose that player's two cards
Tabs above the grid	switch between the two players' ranges
Click a square	list its individual hands, with strategy and value
Edit this range	paint weights, load a preset, or type range notation
Whole tree, Current line	show the full tree or only the path taken and what follows
Click a point in the tree	jump the table to that point
Solve	run the selected algorithm for the number of iterations in the box, or continue it
Race all four	run CFR, CFR+, DCFR and PCFR+ to the same iteration count
Set up the spot	names, pot, stacks, bet sizes, raise sizes, swap seats, random board